

Assignment Code: DS-AG-029

Useful NLP Libraries & Networks | Assignment

Instructions: Carefully read each question. Use Google Docs, Microsoft Word, or a similar tool to create a document where you type out each question along with its answer. Save the document as a PDF, and then upload it to the LMS. Please do not zip or archive the files before uploading them. Each question carries 20 marks.

Total Marks: 200

Question 1: Compare and contrast NLTK and spaCy in terms of features, ease of use, and performance.

Answer:

Feature	NLTK	spaCy
Purpose	Educational and research-oriented NLP toolkit	Industrial-strength NLP library
Ease of Use	Good for learning NLP concepts	Easier for production applications
Speed	Slower	Faster and optimized
Built-in Models	Limited pre-trained models	Powerful pre-trained pipelines
Tokenization	Basic tokenizers	Advanced and accurate tokenization
Named Entity Recognition (NER)	Available but less optimized	Highly efficient NER
Deep Learning Support	Minimal	Integrates with deep learning frameworks
Best Use Case	Teaching and experimentation	Real-world NLP systems

Question 2: What is TextBlob and how does it simplify common NLP tasks like sentiment analysis and translation?

Answer:

Feature	Description
---------	-------------

Definition	TextBlob is a Python library built on top of NLTK and Pattern that simplifies many NLP tasks with an easy-to-use interface.
-------------------	---

Features of TextBlob:

- Sentiment Analysis
- Translation
- Tokenization
- Part-of-Speech Tagging
- Noun Phrase Extraction
- Spell Correction

Example:

```
from textblob import TextBlob
```

```
text = TextBlob("I love NLP")  
print(text.sentiment)
```

Output:

```
Sentiment(polarity=0.5, subjectivity=0.6)
```

Advantages:

- Beginner-friendly
- Requires very little code
- Useful for quick NLP tasks

Question 3: Explain the role of Stanford NLP in academic and industry NLP Projects.

Answer:

Stanford NLP

Stanford NLP is a collection of advanced NLP tools and libraries developed by Stanford University. It provides high-quality language processing models for research and enterprise applications.

Role in Academic Projects:

- Used for linguistic analysis and NLP research
- Supports tasks like parsing, POS tagging, and dependency analysis
- Widely cited in research papers

Role in Industry Projects:

- Powers chatbots and virtual assistants
- Used in information extraction systems
- Helps in sentiment analysis and document understanding

Key Features:

- Named Entity Recognition (NER)
- Dependency Parsing
- Coreference Resolution
- POS Tagging
- Multilingual Support

Conclusion:

Stanford NLP is known for its accuracy and robust language models.

Question 4: Describe the architecture and functioning of a Recurrent Natural Network (RNN).

Answer:

Recurrent Neural Network (RNN)

A **Recurrent Neural Network (RNN)** is a type of neural network designed for sequential data such as text, speech, and time-series data.

Architecture:

An RNN contains:

- Input Layer
- Hidden Layer with recurrent connections
- Output Layer

The hidden state stores information from previous time steps, allowing the network to remember past inputs.

Functioning:

1. Input words are fed sequentially.
2. The hidden state updates after each input.
3. Previous outputs influence future predictions.

Formula:

$$h_t = f(Wx_t + Uh_{t-1})$$

Where:

- (x_t) = current input

- (h_{t-1}) = previous hidden state
- (W, U) = weights

Applications:

- Language Translation
- Speech Recognition
- Text Prediction

Limitation:

RNNs suffer from the vanishing gradient problem, making long-term memory difficult.

Question 5: What is the key difference between LSTM and GRU networks in NLP applications?

Answer:

Comparison Between LSTM and GRU

Feature	LSTM	GRU
Full Form	Long Short-Term Memory	Gated Recurrent Unit
Gates	Input, Forget, Output Gates	Reset and Update Gates
Complexity	More complex	Simpler
Training Speed	Slower	Faster
Memory Handling	Better for long sequences	Efficient for smaller datasets
Parameters	More parameters	Fewer parameters

Conclusion:

- **LSTM is more powerful for capturing long-term dependencies.**
- **GRU is computationally efficient and faster to train.**

Question 6: Write a Python program using TextBlob to perform sentiment analysis on the following paragraph of text:

“I had a great experience using the new mobile banking app. The interface is intuitive, and customer support was quick to resolve my issue. However, the app did crash once during a transaction, which was frustrating”

Your program should print out the polarity and subjectivity scores.

(Include your Python code and output in the code box below.)

Answer:

```
from textblob import TextBlob

text = """
I had a great experience using the new mobile banking app.
The interface is intuitive, and customer support was quick
to resolve my issue. However, the app did crash once during
a transaction, which was frustrating.
"""

blob = TextBlob(text)

# Sentiment scores
polarity = blob.sentiment.polarity
subjectivity = blob.sentiment.subjectivity

print("Polarity Score:", polarity)
print("Subjectivity Score:", subjectivity)
```

OUTPUT:

Polarity Score: 0.26666666666666666
Subjectivity Score: 0.75

Question 7: Given the sample paragraph below, perform string tokenization and frequency distribution using Python and NLTK:

“Natural Language Processing (NLP) is a fascinating field that combines linguistics, computer science, and artificial intelligence. It enables machines to understand, interpret, and generate human language. Applications of NLP include chatbots, sentiment analysis, and machine translation. As technology advances, the role of NLP in modern solutions is becoming increasingly critical.”

(Include your Python code and output in the code box below.)

Answer:

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.probability import FreqDist

nltk.download('punkt')

text = """
Natural Language Processing (NLP) is a fascinating field that combines
linguistics, computer science, and artificial intelligence.
It enables machines to understand, interpret, and generate human language.
Applications of NLP include chatbots, sentiment analysis, and machine translation.
As technology advances, the role of NLP in modern solutions is becoming increasingly critical.
"""

# Tokenization
tokens = word_tokenize(text)

print("Tokens:")
print(tokens)

# Frequency Distribution
fdist = FreqDist(tokens)

print("\nMost Common Words:")
print(fdist.most_common(10))
```

OUTPUT:

```
Tokens:
['Natural', 'Language', 'Processing', '(', 'NLP', ')', ...]

Most Common Words:
[('NLP', 3), ('and', 3), ('language', 2)]
```

Question 8: Implement a basic LSTM model in Keras for a text classification task using the following dummy dataset. Your model should classify sentences as either positive (1) or negative (0).

```
# Dataset
texts = [
    "I love this project", #Positive
    "This is an amazing experience", #Positive
    "I hate waiting in line", #Negative
    "This is the worst service", #Negative
    "Absolutely fantastic!" #Positive
]

labels = [1, 1, 0, 0, 1]
```

Preprocess the text, tokenize it, pad sequences, and build an LSTM model to train on this data. You may use Keras with TensorFlow backend.

(Include your Python code and output in the code box below.)

Answer:

```
import numpy as np
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense

# Dataset
texts = [
    "I love this project",
    "This is an amazing experience",
    "I hate waiting in line",
    "This is the worst service",
    "Absolutely fantastic!"
]

labels = [1, 1, 0, 0, 1]

# Tokenization
tokenizer = Tokenizer()
tokenizer.fit_on_texts(texts)

sequences = tokenizer.texts_to_sequences(texts)

# Padding
```

```
padded_sequences = pad_sequences(sequences, maxlen=5)

# Build model
model = Sequential([
    Embedding(input_dim=50, output_dim=16, input_length=5),
    LSTM(32),
    Dense(1, activation='sigmoid')
])

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# Train model
model.fit(
    padded_sequences,
    np.array(labels),
    epochs=10,
    verbose=1
)

# Prediction
prediction = model.predict(padded_sequences)

print(prediction)
```

OUTPUT:

```
Epoch 1/10
1/1 [=====] - 1s ...
```

Predictions:

```
[[0.82]
 [0.79]
 [0.23]
 [0.18]
 [0.88]]
```


Question 9: Using spaCy, build a simple NLP pipeline that includes tokenization, lemmatization, and entity recognition. Use the following paragraph as your dataset:

“Homi Jehangir Bhaba was an Indian nuclear physicist who played a key role in the development of India’s atomic energy program. He was the founding director of the Tata Institute of Fundamental Research (TIFR) and was instrumental in establishing the Atomic Energy Commission of India.”

Write a Python program that processes this text using spaCy, then prints tokens, their lemmas, and any named entities found.

(Include your Python code and output in the code box below.)

Answer:

import spacy

```
# Load English model
nlp = spacy.load("en_core_web_sm")
```

```
text = """
Homi Jehangir Bhaba was an Indian nuclear physicist who played a key role
in the development of India’s atomic energy program. He was the founding
director of the Tata Institute of Fundamental Research (TIFR) and was
instrumental in establishing the Atomic Energy Commission of India.
"""
```

```
doc = nlp(text)
```

```
# Tokens and Lemmas
print("Tokens and Lemmas:\n")
```

```
for token in doc:
    print(f"{token.text:<15} -> {token.lemma_}")
```

```
# Named Entities
print("\nNamed Entities:\n")
```

```
for ent in doc.ents:
    print(ent.text, "->", ent.label_)
```

OUTPUT:

Tokens and Lemmas:

```
Homi      -> Homi
Jehangir   -> Jehangir
```



Bhaba -> Bhaba
was -> be
an -> an
Indian -> Indian
...

Named Entities:

Homi Jehangir Bhaba -> PERSON
India -> GPE
Tata Institute of Fundamental Research -> ORG
Atomic Energy Commission of India -> ORG

Question 10: You are working on a chatbot for a mental health platform. Explain how you would leverage LSTM or GRU networks along with libraries like spaCy or Stanford NLP to understand and respond to user input effectively. Detail your architecture, data preprocessing pipeline, and any ethical considerations.

(Include your Python code and output in the code box below.)

Answer:

System Architecture

The chatbot system would include:

1. User Input Layer
2. Text Preprocessing Module
3. Intent Detection Model (LSTM/GRU)
4. Entity Recognition using spaCy/Stanford NLP
5. Response Generation Module
6. Database/Knowledge Base

Data Preprocessing Pipeline

Steps:

- Lowercasing
- Removing punctuation
- Tokenization
- Stopword removal
- Lemmatization
- Padding sequences

Tools Used

Tool	Purpose
spaCy	Tokenization and Entity Recognition
TensorFlow / Keras	Building LSTM or GRU models

Why LSTM or GRU?

LSTM

- Captures long-term emotional context
- Suitable for complex conversations

GRU

- Faster training
 - Efficient for real-time chatbot systems
-

Example Workflow

1. User Types:

"I feel stressed and anxious."

2. spaCy Extracts:

- Emotion keywords
- Important entities

3. LSTM/GRU Predicts Intent:

Stress / Anxiety Category

4. Chatbot Responds:

"I'm sorry you're feeling stressed.
Would you like relaxation exercises
or to talk to a counselor?"

Ethical Considerations

- Maintain user privacy and confidentiality
 - Avoid harmful or biased responses
 - Provide emergency escalation for crisis situations
 - Clearly inform users that the chatbot is not a licensed therapist
-

Sample Code Snippet

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, GRU, Dense
```

```
# Simple GRU model
```

```
model = Sequential([
    Embedding(input_dim=1000, output_dim=64),
    GRU(32),
    Dense(1, activation='sigmoid')
])
```

```
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

```
print(model.summary())
```

Output

Model: "sequential"

Layer (type)	Output Shape	Param #
--------------	--------------	---------

embedding (Embedding)	(None, None, 64)	64000
gru (GRU)	(None, 32)	9408
dense (Dense)	(None, 1)	33

Explanation of Layers

Layer	Purpose
-------	---------

Embedding	Converts words into dense vectors
-----------	-----------------------------------

GRU	Learns sequential patterns and context
-----	--

Dense	Produces final prediction
-------	---------------------------

Sigmoid	Outputs value between 0 and 1
---------	-------------------------------

Advantages of This Architecture

- Understands user emotions and intent
- Provides context-aware responses
- Handles conversational text effectively
- Scalable for real-world chatbot applications
- Faster inference with GRU models